



STUDY
MATERIAL
FOR

Open Source Tools

III SEMESTER BCA

2025-2026
SEP Syllabus

**MANGALORE UNIVERSITY COMPUTER
SCIENCE TEACHERS' ASSOCIATION**

UNIT - I

Open Source Software

WHY OPEN SOURCE?

The typical computer user owns lots of software that he bought years ago and no longer uses today. He might have upgraded his computer or changed brands, and then the program wouldn't work any longer. The software might have become old. The program may simply not do what he needs. He may have bought two or more computers and doesn't want to pay for a second copy of the software. Whatever the reason, the software that he paid for years ago is not doing the task today. Does that really need to happen?

What if you had the right to get a free upgrade whenever your software needed it? What if, when you switched from a Mac to a PC, you could switch software versions for free? What if, when the software doesn't work or isn't powerful enough, you can have it improved or even fix it yourself? What if the software was still maintained even if the company that produced it went out of business? What if you could use your software on your office workstation, and your home desktop computer, and your portable laptop, instead of just one computer? You'd probably still be using the software you paid for years ago.

WHAT IS OPEN SOURCE?

Open source or Open-Source Software (OSS) is different from proprietary software. In Open Source, the source code used in the software is available to anyone to examine, evaluate, and adapt. Open source has an important impact on the way many developers view and create software. End users often use the term open source to cover a variety of free and open-source software.

Open source is a development method for software that harnesses the power of distributed peer review and transparency of process. The Open Source Initiative has this for a definition of open source software: The Open Source Definition is a bill of rights for the computer user. It defines certain rights that a software license must grant you to be certified as Open Source.

Examples for open-source software are:

- Linux
- Apache
- BSD
- Wikipedia
- Mozilla.

Programmers feel comfortable contributing to Open Source because they are assured of these rights:

- ▶ The right to make copies of the program, and distribute those copies.
- ▶ The right to have access to the software's source code, a necessary

preliminary before you can change it.

- ▶ The right to make improvements to the program.

These rights are important to the software contributor because they keep all contributors at the same level relative to each other. Everyone who wants to be allowed to sell an Open-Source program, so prices will be low and development to reach new markets will be rapid. Anyone who invests the time to build knowledge in an Open-Source program can support it, and this provides users with the option of providing their own support, or the economy of several competing support providers. Any programmer can tailor an Open-Source program to specific markets in order to reach new customers. People who do these things are not compelled to pay royalties or license fees.

The promises of open source are :

- ▶ Better Quality
- ▶ Higher Reliability
- ▶ More Flexible
- ▶ Lower Cost
- ▶ Transparent

Open source doesn't just mean to access the source code.

The Distribution Terms of open- s o u r c e software must comply with the following criteria:

Free Redistribution

The license shall not restrict any party from selling or giving away the software as a component of an aggregate software distribution containing programs from several different sources. The license shall not require a royalty or other fee for such sale.

Source Code

The program must include source code and must allow distribution in source code as well as compiled form. Where some form of a product is not distributed with source code, there must be a well-publicized means of obtaining the source code for no more than a reasonable reproduction cost- preferably, downloading via the Internet without charge. The source code must be the preferred form in which a programmer would modify the program. Deliberately obfuscated source code is not allowed. Intermediate forms such as the output of a preprocessor or translator are not allowed.

Derived Works

The license must allow modifications and derived works and must allow them to be distributed under the same terms as the license of the original software. provision could allow unscrupulous people to modify their software in ways that would embarrass the original author. They fear someone deliberately making the software perform incorrectly in a way that would make it look as if the author was a poor programmer. Others are concerned that software could be modified for criminal use, by the addition of Trojan horse functions or locally banned technologies such as

cryptography. All these actions, however, are covered by criminal law. A common misunderstanding about software licenses is that they must specify everything, including things like "don't use this software to commit a crime." However, no license has any valid existence outside of the body of civil and criminal law.

Integrity of the Author's Source Code

The license may restrict source code from being distributed in modified form only if the license allows the distribution of "patch files" with the source code for the purpose of modifying the program at build time. The license must explicitly permit distribution of software built from modified source code. The license may require derived works to carry a different name or version number from the original software.

No Discrimination

Against Persons or Groups, The license must not discriminate against any person or group of persons.

To get the maximum benefit from the process, the maximum diversity of people and groups should be equally eligible to contribute to open sources. Therefore, we forbid any open-source license from locking anybody out of the process.

Some countries, including United States, have export restrictions for certain types of software. An OSD-conformant license may warn licensees of applicable restrictions and remind them that they are obliged to obey the law; however, it may not incorporate such restrictions itself.

No Discrimination Against Fields of Endeavor

The license must not restrict anyone from making use of the program in a specific field of endeavor. For example, it may not restrict the program from being used in business, or for genetic research.

Distribution of License

The rights attached to the program must apply to all to whom the program is redistributed without the need for execution of an additional license by those parties.

License Must Not be Specific to a Product

The rights attached to the program must not depend on the program's being part of a particular software distribution. If the program is extracted from that distribution and used or distributed within the terms of the program's license, all parties to whom the program is redistributed should have the same rights as those that are granted in conjunction with the original software distribution.

License Must Not Restrict Other Software

The license must not place restrictions on other software that is distributed along with the licensed software. For example, the license must not insist that all other programs distributed on the same medium must be open-source software.

A version of Ghost Script (a Postscript-rendering program) requires that the media on which it is distributed contain only free software programs. This isn't permitted for Open-Source licenses. Fortunately, the Ghost Script author distributes another (somewhat older) version of the program with a true Open-Source license.

license Must Be Technology-Neutral

No provision of the license may be predicated on any individual technology or style of interface.

OPEN-SOURCE PRINCIPLES

Open Source is a term most applied to software, and it refers to set of principles and practices where by the design and production information is shared openly, without the normal restriction encountered when developing software within a commercial environment where intellectual property is closely guarded.

The brief explanation of each principle is as follows:

Licenses are Free to Use Open-Source Software for Any Purpose Whatsoever

An open-source license may not interfere in any way with the use of the software by licensees. Restrictions on use, such as "*for research and noncommercial purposes only*," are not allowed in open-source licenses. The phrase free to use is also intended to mean, "Without any conditions that would impede use," such as a requirement for the licensee to report uses to the licensor, or to disclose the means or manner of internal uses of the software. Note also that the first word, licensees, means that open-source software is only available under the terms of a license to which each licensee must agree.

Licenses are Free to Make Copies of Open-Source Software and to Distribute them without Payment of Royalties to a licensor

This principle does not mean that a licensor cannot sell open-source software. It merely says that a licensee need not pay the licensor for additional copies he makes himself, even if those copies are distributed to others. As a practical matter, this open-source principle drives the price of mere copies of open-source software toward its marginal cost of production and distribution.

Licenses are Free to Create Derivative Works of Open-Source Software and to distribute them without Payment of Royalties to a licensor

Quality software is built upon the foundations of earlier software. Many advocates of free and open-source software contend that the requirement for open-source licenses to permit the unhindered creation and distribution of derivative works is essential to meet the goal of the intellectual property laws as stated in the U.S. Constitution, "to promote the progress of science and the useful arts." Under this open-source principle, a licensor cannot charge a royalty for the privilege to create and distribute derivative works, or require a licensee to pay a royalty for copies of derivative work that are distributed, or impose any restrictions on the type or character of those derivative works.

Licenses are Free to Access and use the Source Code of Open-Source Software

Source code is written in a human language to instruct a computer how to perform certain functions. Since the source code must be changed to instruct the computer

to perform different functions, access to the source code is essential to make the third open-source principle-the freedom to create derivative works of open source software-a practical reality, Source code is a means to an end, not the end itself. The phrase free to access merely requires the licensor to make source code available to licensees upon request at zero prices, not necessarily to distribute the source code to everyone.

Licenses are Free to Combine Open Source and Other Software

Open-source licenses may not impose conditions or restrictions on other software with which the licensed software is merely combined or distributed. This prevents restrictions regarding what other software can be placed on computer storage media or in computer memory. Open source is one of many possible business and licensing models for software distribution, and customers must be free to select and use those software alternative open source principle does not mean that licensors cannot impose reciprocal upon licensees who create and distribute derivative works, for the activity of creating derivative works is not the same as merely combining software on media or on computers.

OPEN STANDARDS REQUIREMENTS FOR SOFTWARE

An "Open Standard" must not prohibit conforming implementation in open-source software.

The Criteria

To fulfil the Open Standards Requirement, an "Open Standard" must satisfy the following criteria. If an "Open Standard" does not meet the criteria, it will be discriminating against open source developers.

No Intentional Secrets

The standards MUST NOT withhold any detail necessary for interoperable implementation. As flaws are inevitable, the standard MUST define a process for fixing flaws identified during implementation and interoperability testing and incorporate said changes into a revised version or superseding version of the standards to release under terms that do not violate the OSR.

Availability

The standard MUST be free and publicly available (ex. From a stable web site) under royalty-free terms at reasonable and non-discriminatory cost.

Patents

All patents essential to implementation of the standards Must:

- ▶ Be licensed under royalty-free terms for unrestricted use, or
- ▶ Be covered by a promise of non-assertion when practiced by open-source software.

No Agreements

There MUST NOT any requirement for execution of a license agreement, NDA, grant, click-through or any other form of paperwork to deploy conforming implementation of the standard.

No OSR-Incomplete Dependencies

Implementation of the standard MUST NOT require any other technology that fails to meet the criteria of this requirement.

WHERE OSS HAS SUCCEEDED

There are many open-source projects.

- ▶ SourceForge.net, a leading site for open-source development, lists nearly 80,000.
- ▶ A small number of open-source projects are highly successful.
- ▶ Provide widely used alternative to traditional commercial or "closed source" software.
- ▶ Many more projects are used by small communities of experts.
- ▶ The most successful projects tend to be infrastructure software, not applications.

OPEN-SOURCE SUCCESSES

- ▶ More on server than client side
- ▶ More expert users of servers
- ▶ Server operating systems
- ▶ Linux
- ▶ Certain server software
- ▶ Web
- ▶ Mail
- ▶ DNS - So far less so with other server software
- ▶ Database (MySQL rising, less easy to measure)
- ▶ Software for technical users.

FREE SOFTWARE

Free software is software that can be used, studied, and modified without restriction, and which can be copied and redistributed in modified or unmodified form either without restriction, or with restrictions only to ensure that further recipients can also do these things. To make these acts possible, the human readable form of the program (called the source code) must be made available. The source code may be either accompanied by a software license saying that the copyright holder permits these acts (a free software license), or be released into public domain, so that these rights can be automatically held.

Alternative terms for free software have been coined to make the use of "free" less ambiguous. The most common is "open-source software", which has since evolved to refer to a slightly different sense of freedom. Free software is also, known as "software libre", "free, libre and open-source software" ("FLOSS"), and "free and open-source software" ("FOSS").

Free software is distinct from freeware; freeware is proprietary software made available free of charge. One can use, but not study, modify or redistribute freeware.

Since free software may be freely redistributed and generally available at very little cost. Free software business models are usually based on adding value such as support, training, customization, integration, or certification. At the same time, some business models which work with non-free software are not compatible with free software, such as those that depend on a user having no choice but to pay for a license to lawfully use a software product. The free software movement was launched in 1983 to make these freedoms available to every computer user. Software that does not provide these freedoms is referred to as proprietary software or non-free software.

Free software is software that may be modified and redistributed freely by anyone, with no significant restrictions on how the code may be changed, the uses to which it may be put, or the parties with whom it may be shared.

Today, free software is a large body of high-quality code on which much of the Internet depends on critical functions, and it constitutes the core operating system for an increasing number of desktop machines as well.

WHAT IS FREE SOFTWARE?

- ▶ Free software is a matter of liberty, not price. To understand the concept, you should think of free as free speech, not as in free beer.
- ▶ Free software is a matter of the users' freedom to run, copy, distribute, study, change and improve the software.
- ▶ Free in Free Software is referring to freedom, not price. Having been used in this meaning since the 80s, the first documented complete definition appears to be the GNU's Bulletin, vol. 1 no. 1 [I], published February 1986.

Four freedoms define Free Software as

The Freedom to Run the Program, for any Purpose

Placing restrictions on the use of Free Software, such as time ("30 days trial period", "license expires January 1st, 2004") purpose ("permission granted for research and noncommercial use", "may not be used for benchmarking") or geographic area ("must not be used in country X") makes a program non-free.

The Freedom to Study How the Program Works, and Adapt it to Your Needs

Placing legal or practical restrictions on the comprehension or modification of a program, such as mandatory purchase of special licenses, signing of a Non-Disclosure Agreement (NDA) or programming languages that have multiple forms or representation making the preferred human way of comprehending and editing a program ("source code") inaccessible also makes it proprietary (non-free). Without the freedom to modify a program, people will remain at the mercy of a single vendor.

The Freedom to Redistribute Copies So You Can Help Your Neighbor

Software can be copied/distributed at virtually no cost. If you are not allowed to give a program to a person in need, that makes a Program non-free. This can be done for a charge, if you choose.

The Freedom to Improve the Program, and Release Your Improvements to the Public, so that the Whole Community Gets Benefits

Not everyone is an equally good programmer in all fields. Some people don't know how to program at all. This freedom allows those who do not have the time or skills to solve a problem to indirectly access the freedom to modify. This can be done for a charge.

Free Software does not exclude commercial use. If a program fails to allow commercial use and commercial distribution, it is not Free Software. Often Free Software is referred to as "free as in speech, not as in beer", stressing the idea that the Free-software movement is concerned with freedom, not with price.

WHAT ARE SOME EXAMPLES OF FREE SOFTWARE?

Some free software are:

- 1) Operating Systems: Linux, BSD, Darwin and Open Solaris
- 2) GCC Compiler, GDB debugger and C/C++ libraries.
- 3) Servers: BIND name server, Send mail transport, Apache web server and Samba file server.
- 4) Relational Database systems: MySQL and PostgreSQL.
- 5) Programming language: Java, Perl, PHP, Python, Lua, Ruby and Tcl.
- 6) GUI Related: X Windows System, GNOME, KDE, and Xfce desktop environments.
- 7) GRASS: Open Source Geography Information System (GIS).
- 8) OpenOffice.org office suite, Mozilla and Firefox web browser and GIMP graphics editor.
- 9) Typesetting and document preparation system Tex and Latex.
- 10) OGG : Free Software multimedia file type that can function like an mp3 or mp4.
- 11) The package released by the Mozilla Foundation, including the Firefox Browser, and Thunderbird E-Mail client, is Free Software.
- 12) Open Office, the Gamin IM application, the GIMP image manipulation software, the Azures Bit Torrent Client, the VLC Media Player, and the Audacity audio editing software. All of these are Free Software, and you can download them for free right now, use them for as long as you like, and share them with whomever you like.
- 13) First, the Operating Systems: Linux, FreeBSD, and Open Solaris are all

Free Software Operating Systems. Each has several "flavors", often referred as 'distributions', or 'distros' for short.

- 14) The most popular Web Server software, by a large margin, is the Apache Web Server. One the most powerful and versatile Databases is MySQL. They both are not only Free Software but also special hardware to try them out on your local machine.
- 15) The three most used scripting/interpreted Languages are the “P languages” - Perl, PHP, and Python. All of them are Free. You can go to their respective websites and download their interpreters, source codes, specifications, versions documentation, mods, apps, etc., to your heart’s content.

Even proprietary software has Free Software within it. These are commonly libraries that are included within the program that provide some specific function for the program

FREE SOFTWARE LICENSE PROVIDER

Most Free Software is released less than one of the following license agreements:

GNU General Public License

The strict GPL license is the most common, and the most demanding of users of software released under it. The Linux Kernel is released under (version 2 of) this license, as well as the **GNU Compiler Collection (The GCC)**.

GNU Lesser General Public License

This license was originally released as a compromise between the strict GPL license and the more permissive licenses such as the BSD license and the MIT license. It does not carry the "inheritance" element that the GPL license does, which means that commercial software may use software released under this license without having to be released under the GPL, or the LGPL license itself.

BSD Licenses

Sometimes referred to as "BSD-Style" licenses, these licenses are more permissive than the GNU licenses. The original was used for the Berkley Software Distribution (BSD), a Unix-like operating system for which the license was named.

Mozilla License

The Mozilla Public License was developed for software released by the Mozilla Foundation (it was actually developed prior to that, but today that is its primary use). It is a "weak copy-left" license, that has since been adopted, in modified versions, by several other companies and organizations. Mozilla Firefox, Mozilla Thunderbird, and the Mozilla Application Suite are some of the software programs released under this license.

MIT License

This license, which originated at the **Massachusetts Institute of Technology**, is a public domain license. It is one of the most permissive types of licenses, and is the license under which the X Windows System is released.

Apache License

Technically, this license is incompatible with the GPL, since it explicitly allows modifications to the software itself to remain secret and said modifications to be sold.

This does not mean that software released under this license is not Free Software, since it does require the party releasing the software to adhere to the 4 Freedoms. The Apache Web Server is released under this license.

The "as-is" Release Model

This usually means that the released code is simply placed in the public domain, with no restrictions whatsoever apart from not "claiming rights" to that code, and not claiming that the user wrote it. There are actually quite a few dedicated-purpose libraries released under this license, which do something very specific, very efficiently.

FREE SOFTWARE VS. OPEN-SOURCE SOFTWARE

The primary distinction of open-source software is that it's not about freedom; it's about what software does things better.

The movement is often called "**Free and Open-source Software**" (**FOSS**). While many people point to the differences between the two. For the purposes of clarity the abbreviation FOSS will be used when describing issues that apply to all open source software projects, whereas OSS will be used when describing issues that apply only to software with open source licenses that allow future developers to close the source code, and FSS will be used when talking about issues that apply only to software with free licenses requiring future developers maintain the previous style of licensing (if not the license itself).

Critics have said that the term "open source" fosters an ambiguity of a different kind such that it confuses the mere availability of the source with the freedom to use, modify, and redistribute it. Developers have used the alternative terms Free/Open Source Software (FOSS), or Free/Libre/Open-Source Software (FLOSS), consequently, to describe open source software, which is also free software.

The term "Open Source" was originally intended to be trademarkable, however, the term was deemed too descriptive, so no trademark exists. The OSI would prefer that people treat Open Source as if it were a trademark and use it only to describe software licensed under an OSI approved license.

There have been instances where software vendors have labeled proprietary software as "open source" because it interfaces with popular OSS (such as Linux). Open-source advocates consider this to be both confusing and incorrect. OSI certified is a trademark licensed only to people who are distributing software licensed under a license listed on the Open-Source Initiative's list.

Open-source software and free software are different terms for software, which comes with certain rights, or freedoms, for the user. They describe two approaches and philosophies towards free software. Open source and free software (or software libre) both describe software, which is free from onerous licensing restrictions. It may be used, copied, studied, modified and redistributed without restriction. Free software is not the same as freeware, software available at zero prices.

The definition of open-source software was written to be almost identical to the free software definition. There are very few cases of software that is free software but is not open-source software, and vice versa. The difference in the terms is where they place emphasis. "Free software" is defined in terms of giving the user freedom. This reflects the goal of the free software movement. "Open source" highlights that the source code is viewable to all and proponents of the term usually emphasize the quality of the software and how this is caused by the development models which are possible and popular among free and open source software projects.

Free software licenses are not written exclusively by the FSF. The FSF and the OSI both list licenses, which meet their respective definitions of free software. Open source software and free software share an almost identical set of licenses. One exception is an early version of the Apple Public Source License, which was accepted by the OSI but rejected by the FSF because it did not allow private modified versions; this restriction was removed in later version of the license. There are now new versions that are approved by both the OSI and the FSF.

The Open Source Initiative believes that more people will be convinced by the experience of freedom. The FSF believes that more people will be convinced by the concept of freedom.



Fig: Logo of public domain

The FSF believes that knowledge of the concept is an essential requirement, insists on the use of the term free, and separates itself from the open source movement. The Open Source Initiative believes that free has three meanings: free as in beer, free as in freedom, and free, as untellable the problem with the term "open source" is it says nothing about the freedom to modify and redistribute, so people who think that source access without freedom is a sufficient definition use it. This possibility for misuse is the case for most of the licenses that make up Microsoft's "shared source" initiative.

PUBLIC DOMAIN

Public domain comprises the body of knowledge and innovation especially creative works such as writing, art, music, and inventions) in relation to which no person or other legal entity can establish or maintain proprietary interests within a particular legal jurisdiction. This body of information and creativity is part of a common cultural and intellectual heritage, which, in general, anyone may use or exploit, whether for commercial or non-commercial purposes. About 15 percent of all books are in the public domain, including 10 percent of all books that are still in print.

If an item ("work") is not in the public domain, it may be the result of a proprietary interest such as copyright, patent, or other Sui generis right. The extent to which members of the public may use or exploit the work is limited to the extent of the proprietary interests in the relevant legal jurisdiction. However, when copyright,

patent or other proprietary restrictions expire, the work enters the public domain and may be used by anyone for any purpose.

Public Domain Software

Another common form of open source software is software that has entered into the public domain. This is software that is unrestricted by copyright or licenses, and therefore free to use for any purpose. Before the rise of the open source software movement a great deal of software written in academic circles was released into the public domain for peer review. This practice has changed with the rise of a more intentional open source movement, resulting in much software that was once public domain, now being released as FOSS. The public domain is where many resources used in open source projects come from but there are few (if any) major projects that operate totally within the public domain.

DEFINITION: Programs that are uncopyrighted because their authors intended to share them with everyone else are in the public domain. The UNIX community has developed several such programs over the years. Programs in the public domain can be used without restriction as components of other programs.

FOSS DOESNOT MEAN ANY COST

This is a common misunderstanding about FOSS; in no small part because nearly all FOSS programs are available free of charge. For example, when the text editor Emacs was first released Richard Stallman charged from time to time to get copies. Developers have the choice to charge under most FOSS licenses, although they rarely choose to. The only requirement to be a truly FOSS project is that the user provides the source code with licensing the program, and to allow the user to edit that code.

On top of the initial cost of purchasing software, there are other ongoing costs associated with all software. This can come in the form of support agreements, the cost of customization, training costs support personnel and other sources. This is true of both traditional commercial software and FOSS programs. There is a large and active debate about which type of software is more expensive over the long run for large corporations, for individual users there is little which to no matter that FOSS is cheaper by far.

HISTORY

In the beginning (1960's and 1970's) nearly all code was provided as a form of open source. Since you had to get programs to run on many different machines, companies that wanted customers to be able to run their software, they had to provide the source code to compile themselves. Frequently licensing agreements did not encumber this code, or those licenses were never enforced. Often the users would notice flaws in the programs, fix them and provide the improvements back to the publisher at no charge. Unix user groups often shared code with each other as well,

creating an even larger body of code to work from the time most companies believed that hardware would always generate far more profit than software.

At the same time when academics wrote programs as part of their research, they would often release the code into the public domain to others to learn from their work. The first email server, ftp server, and web server were all public domain projects that were created by academics and shared as public domain software.

Over time the hardware became more standardized. This allowed software publishers to tighten up on what segments of code they would allow their users to see and edit. This tightening started to frustrate users, especially academics; since they could no longer fix the problem they found, and from time to time found that they couldn't use the hardware or software they wanted to. This trend mostly continued in mainstream software, strengthened by the rise of Microsoft.

During the late 1970's, many small businesses attempted to survive writing software for the new microcomputers. None were able to rely on hardware sales or after sale support contracts to subsidize software development. Few of these people, whether from an academic tradition of freely distributed software or not, were willing to make their products available without fee. Perhaps most famously, Bill Gates (a co-founder of Microsoft) realized that software, not hardware, held the prospect of being the greatest profit source in the new world of computing. When IBM needed an operating system for their personal computers instead of writing their own, Microsoft licensed an early version of DOS, but retained the licensing rights for other hardware manufacturers.

In this sea of activity, the FOSS movement primarily grew out of two places the Berkeley Software Distribution licenses from the University of California and Richard Stallman's Free Software Foundation Gnu Public License.

BSD

During the 1970s, after Ken Thompson of Bell Labs spent a year teaching at the University of California at Berkeley, several research groups there began to develop software for the UNIX operating system. And, as Bell Labs had no plans to port the system to the then new DEC VAX computer system, the Computer Science Research Group at Berkeley undertook the port. That port became very popular at other universities, and the CSRG began distribution of it, with their modifications and additions, under the name Berkeley Software Distribution. The researchers at Berkeley frequently found ways to improve or add to the existing versions of UNIX. Bell Labs accepted some of the BSD changes and passed on others. Over time the CSRG found ways to improve the basic aspects of UNIX (e.g., the file system). Concurrently, AT&T made improvements of their own, causing the two versions to move in different (although similar) directions. As development progressed on the Berkeley version of UNIX, it grew further away from the work that AT&T was doing on their version (the last version of which was System V). In the early 1990s there was a major legal skirmish over the Berkeley Software Distribution (BSD) between

AT&T, the University of California at Berkeley, and the Unix System Laboratories (USL), to which AT&T had turned over the marketing and development of UNIX. The suit resulted in most of BSD being found to be unencumbered by copyright restrictions by those who controlled the original Bell Labs code. By the mid-1990s, the CSRG released 4.4BSD-Lite, Release 2 as their final product as they were closed down by the Regents of the University of California. The BSD releases were the root of the various versions of BSD that are common today, most notably FreeBSD, Net BSD and Open BSD. Throughout its development, BSD was distributed in an open manner, resulting in contributions from many different contributors.

The Free Software Foundation and the GNU Project

In 1983, Richard Stallman (then of MIT) became frustrated with the growing commercialization of the computer development work that had been done at MIT, and with the increasing limitations imposed on software users. After a time, he began to create software that gave control to users. His vision was to create an entire operating system totally free of the restrictions being imposed by proprietary licensing. His first major software development effort (even before he became disenchanted with trends in the software industry free. To. was the text editor, GNU Emacs. His next was a LISP system, a wick compiler, and finally the GNU project.

In 1985 Stallman founded the **Free Software Foundation (FSF)** to help generate support for the GNU Project. FSF has grown into one of the most important organizations in the FOSS movement. While the primary FSF mission continues to be the completion of the GNU operating system, FSF has also taken on the role of "free-software evangelist" by protecting B and supporting free software. FSF also holds the copyrights too much of the source code written for the GNU project, ensuring that it always remains freely available to users.

Free Software Foundation

The Free Software Foundation (FSF) is a non-profit corporation founded by Richard Stallman on 4 October 1985 to support the free software movement, a copy left-based movement which aims to promote the universal freedom to distribute and modify computer software without restriction. The FSF is incorporated in the Commonwealth of Massachusetts, United States of America.

From its founding until the mid-1990s, FSF's funds were mostly used to employ software developers to write free software for the GNU Project. Since the mid-1990s, the FSF's employees and volunteers have mostly worked on legal and structural issues for the free software movement and the free software community. Being consistent with its goals, only free software is used on all of the FSF's computers.

GNU Project: The GNU Project is free software, mass collaboration project, announced in 1983 by Richard Stallman. It initiated the GNU operating system, software-development for which began in January 1984. GNU is a recursive acronym

that stands for "GNU's Not Unix".

The founding goal of the project was, in the words of its initial announcement, to develop **"a sufficient body of free software to get along without any software that is not free."** To make this happen, the GNU Project began working on an operating system called GNU. This goal of making a free software operating system was achieved in 1992 when the last gap in the GNU system, a kernel, was filled by a third-party Unix-style kernel called "Linux" being released as free software. Current work of the GNU Project includes software development, awareness building, and political campaigning.

Berkeley Software Distribution

Berkeley Software Distribution (BSD), sometimes called Berkeley Unix, is the UNIX derivative by the University of California Berkeley, starting in the 1970s. Historically, BSD has been considered as a branch of UNIX- 'BSD UNIX', because it had shared the initial code base and design with the original UNIX by AT&T and collaborated on the development in the pioneer days of UNIX. It was widely identified with the versions of UNIX available for workstation-class systems, which can be attributed to the ease with which it could be licensed and the familiarity it found among the founders of many technology companies during the 1980s. The familiarity often came from using similar systems-notably DEC's ULTRIX and Sun Microsystems SunOS-during their education. Though BSD itself was largely superseded by the System V Release 4 and OSF/1 systems in the 1990s (both of which incorporated BSD code), the modified code base as open source-mostly derived from 4.4BSD-Lite have seen increasing use and development recently.

Today, the term of "BSD" is often non-specifically used to refer to any of the BSD descendants, e.g., FreeBSD, NetBSD or OpenBSD, which forms a branch of Unix-like operating systems.

WHY IS "FREE SOFTWARE" BETTER THAN "OPEN SOURCE"

"Open Source" misses the point of Free Software is an updated version of this article. While free software by any other name would give you the same freedom, it makes a big difference which name we use: different words convey different ideas. In 1998, some of the people in the free software community began using the term "open-source software" instead of "free software" to describe what they do. The term "open source" quickly became associated with a different approach, a different philosophy, different values, and even a different criterion for which licenses are acceptable. The Free Software movement and the Open Source movement are today separate movements with different views and goals, although we can and do work together on some practical projects.

The fundamental difference between the two movements is in their values their ways of looking at the world. For the Open Source movement, the issue of whether software should be open source is a practical question, not an ethical one. As one person put it, "Open source is a development methodology; free software is a social

movement." For the Open source-movement, non-free software is a suboptimal solution. For the Free Software movement, non-free software is a social problem, and free software is the solution.

DIFFERENCES BETWEEN Closed Source Open-source S/W

Proprietary/Closed Source S/W	Open-Source S/W
Licensors distribute object code only; source code is kept as a trade secret.	Licensors distribute source code
Modifications are prohibited	Modifications are permitted
All upgrades, support and development are done by the licensor	The Licensee may do own development and support or hire any third party to do it
Fees are for the software license, maintenance and upgrade.	Fees, if any are for integration, packing, support and consulting
Sublicensing is prohibited or it is very limited right.	Sublicensing is permitted. The Licensee may have to distribute the source code to program and modification
Microsoft Windows is an example of Closed Source S/W	Linux is an example of Open-Source S/W

WHY DO SOME COMPANIES USE OPEN-SOURCE SOFTWARE?

Some companies use open-source software because:

- Cost savings
- Access and broad rights to source code
- Access to skilled community of developers
- Ability to define and expedite new development.

WHY DO SOME COMPANIES DON'T USE OPEN SOURCE SOFTWARE

Some companies don't use open-source software because

- Migration costs
- Uncertainty about open development model
- Lack of IP warranties and indemnities
- Copy left provisions of GPL, LGPL and similar licenses.

UNIT - 2

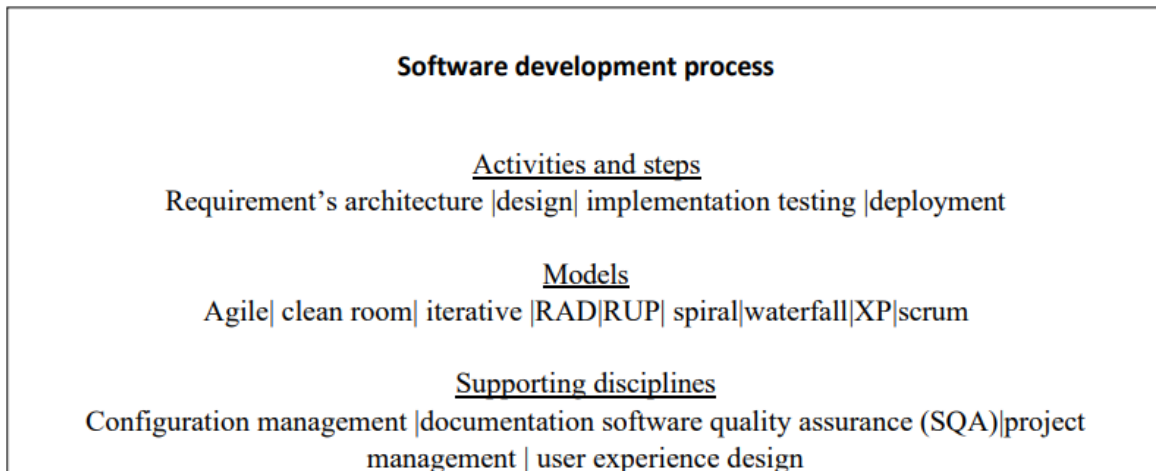
PROGRAMMING TOOLS AND TECHNIQUES

Introduction to Open-Source Software Development:

Open-source software development is the process by which Open-Source Software, or similar software whose source is publicly available, is developed by an open-source software project.

Open-Source Software Development process:

Open-source software development is the process by which open-source software, or similar software which source code is publicly available, is developed. These are software products available with its source code and under the open source license to study, change and improve its design. Examples of popular open-source software products are Mozilla Firefox and the OpenOffice.org suite. The open-source software development method is very unstructured because no clear development tools, faces.



Usually software development involves following process:

Step #1. Analysis and Planning. The first and most crucial step is defining the objectives and core functions of that new product needs. ...

Step #2. Design. ...

Step #3. Implementation (Coding) ...

Step #4. Testing. ...

Step #5. Deployment and Maintenance.

Analysis and Planning:

In open-source projects, this step involves defining the objectives and core functions of the software, just like in proprietary development. However, in the open-source context, this planning often happens collaboratively within the community. Contributors may discuss the project's goals, target audience, feature priorities, and other considerations openly. Community feedback and input can heavily influence the planning process

Design: The design phase involves creating the architecture and high-level design of the software. In open-source projects, this phase may involve collaborative discussions and design proposals within the community.

Implementation (Coding): In this phase, developers write code to implement the design. In opensource projects, multiple contributors may work simultaneously on different parts of the codebase. Collaboration platforms like GitHub facilitate this process by allowing developers to fork the repository, work on changes in their own branches, and submit pull requests to propose changes for review and inclusion into the main codebase.

Testing: Testing is crucial to ensure the quality and reliability of the software. Open-source projects often employ a combination of automated testing and community-driven testing.

Deployment and Maintenance: Once the software is tested and deemed ready for release, it can be deployed for users to use. In the open-source context, deployment may involve making the software available for download through distribution channels like package repositories or official project websites. Maintenance involves ongoing support, bug fixes, and updates to address issues and improve the software.

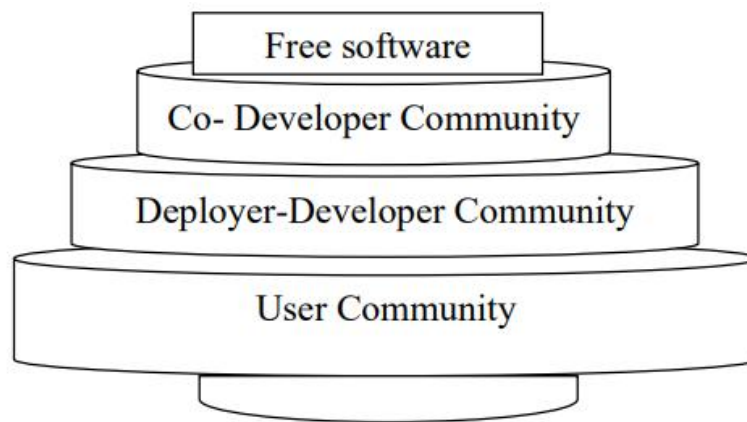
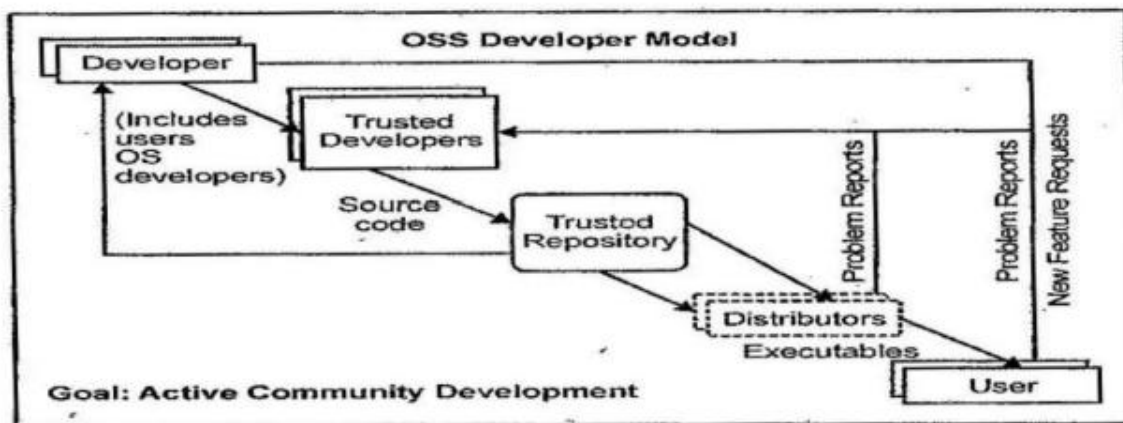


Figure: Open-Source Community

Process data model for open-source software development:

Open-source software development can be divided into several phases. A diagram displaying the process- data structure of open-source software development is shown on the right. In this picture, the phases of open-source software development are displayed, along with the corresponding data elements.



Starting an open-source project: There are several ways in which work on an open-source project can start: An individual who senses the need for a project announces the intent to develop a project in public.

A developer working on a limited but working code base, releases it to the public as the first version of an open-source program. The source code of a mature project is released to the public. A well-established open-source project can be formed by an interested outside party. It's a common mistake to start a project when contributing to an existing similar project would be more effective. To start a successful project it is very important to investigate

what's already there. The process starts with a choice between the adopting of an existing project, or the starting of a new project. If new project is started, the process goes to the Initiation phase. If an existing project is adopted, the process goes directly to the Execution phase.

Process of Starting an Open-Source Project

1. Finding the Need for a Project

Every open-source project begins because someone identifies a need. This could happen in several ways:

- **Idea announcement:** An individual announces publicly that they plan to create a project to solve a problem.
- **Releasing early code:** A developer has a small, working version of a program and releases it so others can help improve it.
- **Opening an existing mature project:** A private or closed-source project is made public so the community can use and improve it.
- **External initiation:** An interested outsider creates a new project to meet a demand that isn't currently addressed.

2. Investigating Existing Solutions

Before starting something new, it's essential to check if similar projects already exist.

- If there's already a well-maintained, active project, it's usually more effective to join and contribute to it rather than starting from scratch.
- This avoids duplicate work and helps improve existing solutions faster.

3. Choosing the Path

After the investigation, there are two possible directions:

- **Adopt an Existing Project:** If you find a similar but inactive or abandoned project, you can take it over and continue development.
- **Start a New Project:** If no suitable project exists, create a new one from scratch.

4. Initiation Phase *(For New Projects)*

If you decide to start fresh:

- Define Goals: Clearly explain what your project will do and who will benefit from it.
- Select a License: Choose an open-source license (like MIT, GPL, Apache) to define usage rights.
- Set Up Tools: Create a code repository (GitHub, GitLab), documentation, and issue tracking system.
- Prototype: Build a small working version to demonstrate the concept.

5. Execution Phase

This is where the main development happens:

- Write and improve code.
- Accept contributions from other developers.
- Fix bugs and add new features.
- Communicate actively with users and contributors to keep the project alive.

6. Maintenance and Growth

Once the project is running:

- Keep it updated to remain relevant and secure.
- Improve and expand documentation for new users.
- Attract and support new contributors.
- Listen to user feedback and make improvements.

Participants in OSS Development Projects:

Participants in OSS development projects fall into 2 broad categories, the core and the peripheral. The core, or inner circle, are developers who modify the primary code that constitutes the project. The peripheral usually consists of users of the software. They report bugs, submit fixes and suggest changes.

The participants can be divided into the following.

- Project leaders who have overall responsibility(core). Most of them might have been involved in coding the first release of the software. They control the overall direction of individual projects.

- Volunteer developers (core or periphery), who do actual coding for the project. These include.
 - Senior members with broader overall authority.
 - Peripheral developers producing and submitting code fixes.
 - Occasional contributors.
 - Maintainers who work on different aspects of the project.
- Everyday users (periphery) who perform testing, identify bugs, deliver bug reports etc
- Posters (periphery) who participate frequently in news groups and discussions but do not do any coding.

Tools used for open-source development:

The tools used for open-source development are,

- Communication channels.
- Software engineering tools
- Testing tools
- Package management
- Automated tests

Communication channels – developers and users of an open-source project are not all necessarily working on the project in proximity. They require some electronic means of communications. Email is one of the most common forms of communication among open source developers and users. Open electronic mailing lists are used to make sure. Email messages are delivered to all interested parties at once. This ensures that at least one of the Members can reply to it. To communicate in real time. Many projects use an instant messaging method. Web forums have recently become a common way for users to get help with problems they encounter when using an open source product. Wikis have become common as a communication medium for developers and users.

Software engineering tools- Version control systems. In OSS development, the participants, who are mostly volunteers, are distributed amongst different geographic regions, so there is need for tools to aid

participants to collaborate in the development of source code. Concurrent versions system CVS is a prominent example of the source code collaboration tool being used in OSS projects. CVS helps manage the files and codes of the project when several people are working on the project at the same time.

CVS(Concurrent Version System) allows several people to work on the same file at the same time. This is done by moving the file into the users directories and then merging the files when the users are done.

CVS also enables one to easily retrieve a previous version of the file. The subversion revision control system SVN was created to replace CVS. It is quickly gaining ground as an OSS project version control system.

Testing tools : Since OSS projects undergo frequent integration, tools that help automate testing during system integration are used. An example of such tool is tinderbox. **Tinderbox** enables participants in an OSS project to detect errors during system integration. Tinderbox runs a continuous build process and informs users about the source code that have issues. And on which platforms these issues arise.

Furthermore, this tool identifies the author of the offending code. The author is then held responsible for ensuring that error is resolved. Mainly because normal testing tools are quite expensive, open source testing tools are gaining popularity. A debugger is a computer program that is used to debug other programs.

GNU Debugger is an example of the debugger used in open-source software development.

A memory leak tool or memory debugger is a programming tool for finding memory leaks and buffer overflows. **Validation tools** are used to check if pieces of code confirm to a specified syntax. An example of a validation tool is **LCLint, now called Splint**.

Package management:

A package management system is a collection of tools to automate the process of installing, upgrading, configuring, and removing software packages from the computer. Package Management Tools help in installing, updating,

removing, and managing software packages efficiently.

Automated tests.

Software testing is an integral part of open source development. Automated Testing Tools are used to execute test cases automatically without manual intervention. They help in software quality assurance by detecting bugs, performance issues, and security vulnerabilities. Examples of automated testing tools include Selenium WebDriver for web application testing, Appium for mobile application testing.

Software Engineering methodology:

Software engineering methodology refers to a structured and organized set of practices, processes, and principles used to design, develop, test, and maintain software systems. It provides a disciplined approach to software development, helping teams manage complexity, improve quality, and meet user requirements effectively.

1. Waterfall Model:

The Waterfall model is one of the earliest software development methodologies. It follows a linear and sequential approach, where each phase—requirements gathering, design, implementation, testing, deployment, and maintenance—is completed one after another. Once a phase is finished, you cannot go back to it. It's best suited for projects with well-defined requirements and low uncertainty.

2. Agile:

Agile is a flexible, iterative, and incremental approach to software development. Work is divided into short cycles called sprints. Agile encourages customer collaboration, early delivery, and continuous improvement. It allows teams to adapt quickly to changing requirements.

3. Spiral Model: The Spiral model blends the Waterfall model's structure with iterative development and risk analysis. Development occurs in a spiral shape, with each loop representing a phase: planning, risk analysis, engineering, and evaluation. It's suitable for large, complex, and high-risk projects where frequent reassessment is needed. Choosing a software license

is a critical decision in software development, especially for open-source projects. A software license defines how others can use, modify, distribute, or share your code. The right license ensures your legal rights are protected while aligning with how you want your software to be used.

Different types of open-source projects

- ▶ **Software programs and libraries:** These are standalone pieces of code that serve a specific purpose or fill a particular need. Examples include the Linux kernel, Firefox web browser, and the openoffice.org office suite.
Firefox Web Browser: Firefox is an opensource web browser developed by Mozilla. The source code is publicly available on Mozilla's GitHub. Anyone can contribute to fixing bugs, improving performance, or adding new features.
- ▶ **Distributions:** These are collections of software published from the same source with a common purpose. Operating systems like Debian, Fedora Core, and Mandriva are examples. Distributions can also include collections of software for specific purposes. In the context of Linux, a Linux distribution is an operating system that includes:

Examples of Linux Distributions:

Debian (GPL-licensed, known for stability).

Fedora Core (Uses the GPL and open-source licenses, focuses on innovation).

Operating system projects: Some open-source projects, like the BSD derivatives, maintain the source code of an entire operating system and its core components in one revision control system. • These projects develop the entire system as a single team, closely integrating their tools. Example: BSD Derivatives the Berkeley Software Distribution (BSD) derivatives, such as FreeBSD, OpenBSD, and NetBSD, are examples of open-source operating system projects.

Book or standalone document projects: These are open-source documentation projects that provide guides, manuals, or references for software and technology. Unlike traditional software projects, these focus on writing and maintaining documentation rather than coding applications.

They help users and developers understand, install, and use open-source software.

Introduction to License and Copyright:

License is a permission or right granted to engage in some act without which the act might be otherwise unlawful. The term license also refers to the document that specifically describes the permissions and rights.

Common licensing practices:

The **GNU General Public License (GPL)** is a type of copyleft license, which means that any software developed using GPL-licensed code must also be released under the same license if it is distributed. One of its main features is that it requires the source code to be made available whenever the software is shared or published. This ensures that users have the freedom to study, modify, and redistribute the software. Additionally, the GPL helps preserve software freedom by making sure that all derivative works carry the same rights as the original. This license is commonly used in projects that aim to keep software free and open for everyone, such as the Linux operating system.

GNU Lesser General Public License (LGPL):

The GNU Lesser General Public License (LGPL) is a more permissive version of the GPL, designed primarily for software libraries. Like the GPL, it ensures that users can access, study, and modify the source code. However, it allows proprietary software to use LGPL-licensed libraries without having to release the proprietary software's source code. If changes are made directly to the LGPL-covered library itself, those changes must be shared under the same license when distributed. This license strikes a balance between encouraging open-source collaboration and allowing broader compatibility with closed-source applications.

BSD License:

The BSD (Berkeley Software Distribution) License is a permissive open-source license that allows software to be used, modified, and redistributed with minimal restrictions. It permits the code to be incorporated into both open-

source and proprietary projects. The main requirements are to retain the original copyright notice and disclaimers in the redistributed code. There's no obligation to disclose source code, even in modified versions. This makes the BSD license attractive for developers who want maximum freedom in how their code is used.

Apache License:

The Apache License is a permissive open-source license maintained by the Apache Software Foundation. It allows users to freely use, modify, and distribute software, including in proprietary products. Like the BSD license, it has few restrictions, but it also includes explicit terms for granting patent rights from contributors to users. This helps protect against patent litigation. Additionally, it requires modified files to carry notices of change, which helps track alterations over time. The Apache License is widely used by organizations that want to encourage open development while maintaining legal clarity and protection.

Factors to be Considered When Choosing any License:

1 Level of Freedom You Want to Grant

- Do you want others to be able to use your code freely, even in closed-source?
- Or do you want to ensure that any derivative works remain open-source?

2 Type of License:

- Permissive Licenses (e.g., MIT, BSD, Apache): Allow almost any use, including commercial and closed-source.
- Copyleft Licenses (e.g., GPL): Require derivative works to be released under the same license, keeping the software open-source.

3. Compatibility with Other Licenses

If you're using third-party libraries, make sure your license is compatible with theirs.

4. Community and Business Goals

For wider adoption and commercial use, a permissive license may be preferred.

What is Copyright? & Copyleft?

Copyright: Copyright is a legal right that gives the creator of original work (like software, books, music, or art) the exclusive right to use, distribute, reproduce, and modify their work for a certain period of time. It automatically applies as soon as the work is created and fixed in a tangible form.

In software, copyright means that only the author (or rights holder) can legally:

- ▶ Copy or distribute the software
- ▶ Modify or create derivative works
- ▶ License it to others under specific terms

Copyleft:

Copyleft is a concept that uses copyright law to ensure that a work and its derivatives remain free and open. It allows anyone to use, modify, and distribute the work—but only under the same license terms. Copyleft is commonly used in open-source licenses, like the GNU General Public License (GPL). It ensures that any modified versions of the software are also shared with the same freedoms, preventing someone from taking open-source code and turning it into proprietary software.

Patent law:

A patent is a legal right granted by a government to an inventor, giving them the exclusive right to make, use, sell, or distribute an invention for a specific period usually 20 years from the filing date.

In exchange, the inventor must publicly disclose the details of the invention so others can learn from it. Patents are a key part of intellectual property law, designed to encourage innovation by rewarding inventors for their creations. To be eligible for a patent, an invention must be new, represent a significant improvement or difference from existing products or ideas, and have a practical application. Once granted, a patent allows the inventor to prevent others from making, using, or selling the invention without permission.

However, it does not give the inventor the right to use the invention themselves if it infringes on another existing patent.

There are six laws of patent:

1. Effects
2. Enforcement
3. Ownership
4. Governing laws
5. Application
6. Economics Ownership

Patent law determines who owns the rights to an invention. Typically, the inventor is the initial owner, but ownership can be transferred (e.g., to an employer if the invention was made during work). Clear ownership is essential for enforcement and licensing.

Application: The patent application process involves filing a detailed description of the invention with a national or international patent office. The application is examined to ensure the invention is novel, non-obvious, and industrially applicable. If it meets all criteria, a patent is granted.

Enforcement: Patent holders have the legal right to enforce their patents through civil litigation. They can sue individuals or companies that use their invention without permission, seeking remedies such as injunctions, damages, or royalty payments. **Governing Laws:** Patent law is governed by national laws (e.g., the U.S. Patent Act, Indian Patent Act) and international agreements, such as the Patent Cooperation Treaty (PCT) and TRIPS Agreement (WTO). These laws standardize patent protection across countries and provide legal frameworks.

Ownership

Ownership in patent law determines who has the legal rights to an invention. Generally, the inventor is the original owner of the patent rights. However, ownership can be transferred to another party, such as an employer, especially if the invention was created as part of employment duties or under a contractual agreement. Clearly defining ownership is crucial because only

the legal owner has the authority to license, sell, or enforce the patent against infringers. Without proper ownership documentation, enforcing rights in court becomes difficult.

Governing Laws

Patent law is governed by a combination of national legislation and international agreements. Each country has its own laws—for example, the United States follows the U.S. Patent Act, while India follows the Indian Patent Act, 1970. At the global level, treaties such as the Patent Cooperation Treaty (PCT) and the Trade-Related Aspects of Intellectual Property Rights (TRIPS) Agreement aim to standardize and simplify patent protection across multiple countries. These frameworks ensure inventors can protect their work internationally while following consistent rules.

Economics

From an economic perspective, patents encourage innovation by rewarding inventors with exclusive rights for a limited period, allowing them to recover research and development costs. Patents can provide a strong competitive advantage in the market by enabling premium pricing, brand recognition, and blocking competitors from using the same technology. They also create revenue opportunities through licensing and royalty agreements.

Introduction to Bug trackers and Task lists:

Bug Trackers: A bug tracker (or issue tracker) is a software tool used to identify, record, monitor, and manage bugs (errors or defects) in a software project. It helps developers track the status of bugs from discovery to resolution. Each bug typically includes a description, severity level, status, assigned developer, and a history of updates.

Key benefits of bug trackers:

- ▶ Centralized bug reporting
- ▶ Improved collaboration between developers and testers
- ▶ Prioritization of issues based on severity
- ▶ Clear tracking of bug resolution progress

Popular bug tracking tools include Jira, Bugzilla, Redmine, and GitHub Issues.

Key features of bug trackers typically include:

1. **Issue Reporting:** Users can submit bug reports detailing the nature of the issue, steps to reproduce it, and any other relevant information.
2. **Issue Tracking:** Each reported issue is assigned a unique identifier and tracked throughout its lifecycle, from submission to resolution.
3. **Status Tracking:** Bug trackers allow users to track the status of issues, including whether they are new, assigned, in progress, resolved, or closed.
4. **Priority and Severity:** Users can assign priorities and severity levels to reported issues to indicate their importance and impact on the software.
5. **Assignment and Ownership:** Issues can be assigned to specific team members responsible for resolving them.

Task Lists:

A task list is a simple tool for keeping track of what needs to be done in a project. It includes tasks such as coding features, writing documentation, testing components, or updating systems. Tasks are often broken down into smaller items and assigned to team members with due dates and priority levels.

Key benefits of task lists:

- ▶ Organized workflow and planning
- ▶ Improved time and resource management
- ▶ Easier delegation of work among team members
- ▶ Tracking project progress and deadlines.

Expand NPL and MPL.

- ▶ NPL – Netscape public licence.
- ▶ MPL – Mozilla public license.

Expand SQA and FDL. • SQA – Software quality assurance. • FDL – free

documentation licence.

Expand GDB and OSI.

- ▶ GDB – GNU Debugger
- ▶ OSI – Open source initiative.

Expand RPM and APT.

- ▶ RPM – Red Hat Package Manager
- ▶ APT – Advanced Packaging Tool

What is UML diagram?

- ▶ UML diagrams are graphical representations of various aspects of software systems, such as classes, objects, relationships, interactions, and behaviors.
- ▶ These diagrams help developers and stakeholders understand the structure, behavior, and architecture of the software being developed.
- ▶ ArgoUML is an open-source developer tool that helps in creating complex designs and diagrams. ArgoUML supports various types of UML diagrams, including:
 - ▶ Class diagrams
 - ▶ Use case diagrams
 - ▶ Sequence diagrams
 - ▶ State diagrams
 - ▶ Activity diagrams

Features of ArgoUML:

- ▶ ArgoUML is written in Java, making it platform-independent. It can run on different operating systems, including Windows, macOS, and Linux.
- ▶ Design tools often support collaboration features, allowing multiple users to work on the same set of diagrams concurrently. This facilitates collaboration among team members, stakeholders, and clients, enabling them to discuss and refine design decisions in real-time.
- ▶ ArgoUML provides an intuitive graphical user interface (GUI) that allows users to create, edit, and manipulate UML diagrams easily. The

interface includes tools for drawing, selecting, moving, resizing, and connecting elements in the diagrams.

- ▶ **Code Generation:** ArgoUML includes functionality for generating code from UML models. Users can generate code in various programming languages, including Java, C++, and others, based on their UML class diagrams.
- ▶ **Documentation:** ArgoUML allows users to annotate UML diagrams with additional information, such as descriptions, notes, constraints, and design decisions. This documentation helps in communicating design decisions to other team members, stakeholders, and future maintainers of the software.

Bootstrap:

In web design, "Bootstrap" refers to a popular front-end framework originally developed by Twitter. It's a collection of HTML, CSS, and JavaScript tools designed to help developers create responsive and mobile-first websites and web applications more efficiently.

Features of Bootstrap:

- ▶ **Open Source:** Bootstrap is an open-source project, licensed under the MIT License, which means it is free to use, modify, and distribute. This encourages collaboration and innovation within the development community.



- ▶ **Pre-styled Components:** Bootstrap provides a collection of pre-styled components such as buttons, forms, navigation bars, dropdowns, alerts, and more. These components help developers quickly build consistent and visually appealing UIs.
- ▶ **Browser Compatibility:** Bootstrap is designed to be compatible with

modern web browsers, ensuring consistent behavior and appearance across different platforms and devices.

- ▶ **Community Support:** Being one of the most widely used front-end frameworks, Bootstrap has a large and active community of developers

The structure of the Bootstrap package typically includes folders such as:

- ▶ **CSS:** Contains the compiled CSS files, including both the standard Bootstrap styles and any customizations you may have made.
- ▶ **JS:** Contains the JavaScript files for Bootstrap components and plugins.
- ▶ **Fonts:** Contains font files used by Bootstrap, such as Glyphicons or any custom font files you may include.
- ▶ **Documentation:** Contains documentation and examples for using Bootstrap.

UNIT 3

APACHE

The Apache Web Server is also referred to as Httpd (Hyper Text Transfer protocol daemon), is an open source software which is available under Apache licence version 2. It was developed by a group of programmers. The first version of the Apache was developed in 1995, using the NCSA (National Center for Supercomputing Applications).

This software is used by 60% of the web domains. Twenty volunteer programmers called themselves the Apache group was involved in the core development of this Web Server. The source code of the software is available for free and users can adapt the software to other specific needs.

The newer versions run under the Windows, OS/s and other platforms. It is also called the patchy server as it was developed from the NCSA code along with a variety of patches. Open Source software development of the Apache Web Server is important to explain the formation of the Apache Foundation.

History of Apache

- ▶ The first version of the Apache web server was created by Robert McCool, who was heavily involved with the National Center for Supercomputing Applications web server, known simply as NCSA Https.
- ▶ When Rob left NCSA in mid-1994, the development of https stalled, leaving a variety of patches for improvements circulating through e-mails.
- ▶ Rob McCool was not alone in his efforts. Several other developers helped form the original "Apache Group".
- ▶ Version 2 of the Apache server was a substantial re-write of much of the Apache 1.x code, with a strong focus on further modularization and the development of a portability layer, the Apache Portable Runtime.
- ▶ The Apache 2.x core has several major enhancements over Apache 1.x. These include UNIX threading, support for non-Unix platforms (such as Microsoft Windows), a new Apache API, and IPv6 support. The first alpha release of Apache 2 was in March 2000, with the first general availability release on 6 April 2002.
- ▶ Version 2.2 introduced a new authorization API that allows for more flexibility. It also features improved cache modules and proxy modules.

Apache Software foundation (ASF)

The Apache group was the forms the Apache Software foundation which is non- profitable corporation in June 1999. The foundation has management which includes Board of Directors and the corporation for yearly election by the members. There are a set of officers appointed by the board to manage the day to day operations administer the ASF projects. There is a team of experts who contribute to the projects and this process of collaboration involves certain guidelines.

The purpose of the foundation is as follows:

1. Ability to provide for software development which will include aspects such as open, collaborative approach by the supply of

hardware, infrastructure for business and communication.

2. Provision of legal entity which is independent where individuals or companies can provide their resources for the benefit of public.
3. Provide a sanctuary for individuals from legal suits who are working on the Foundation's Projects
4. Protection of the Brand Apache from abuse by other organizations.

Features of Apache

1. Support server-side programming language like perl, python, Tel, and PHP.
2. It supports authentication modules such as SSL and TLS support (mod_ssl),
3. Supports custom log files (mod_log_config), and filtering support (mod_include and mod_ext_filter).
4. It supports Popular compression methods like mod_gzip, to reduce of the size (weight) of web pages served over HTTP.
5. Virtual hosting allows one Apache installation to serve many different actual websites.
6. Apache features configurable error message DBMS-based authentication databases, and content negotiation.
7. It is also supported by several Graphical User Interfaces (GUIs) which permit more intuitive configuration of the server.

Apache Web server

The Apache server supports the languages such as Perl, Python, PHP. Additional functionality can be added which includes support for authentication processes, language support and other features.

Examples: modules which support authentication are mod_access, mod_auth, mod_digest, and mod_auth_digest.

SSL(Secure Socket Layer)/TLS (Transport Layer Security) can be provides modules such as URL rewriting (mod_rewrite), custom.

The Apache server supports CGI(Common Gateway Interface), SSI(Server Side Includes) for authentication. After the launch of the web server in April 1996 several versions were released which made changes to the configuration of the software.

For example the latest version Apache 2.0 provides the users with simple to use functionality, IPv6 and Unix support, multi-language ,error messaging and better compilation.

Apache 2.2 provides proxy caching modules, SQL support and user friendly authentication systems. Commercial software use their won HTTP server development with an Apache base, also complying with the Licensing agreements. There is however a problem that the Apache HTTP server which comes from the Apache foundation is used by commercial vendors as their own product. Apache foundation highlights the problem by stating that there is no “Vendor Apache Product”.

Apache web servers can be supported by modern **operating systems such as Windows NT(New Technology), Unix** .The versions provide improved features of authentication, proxy support, Catching, smart filtering, security and bug fix releases.

Apache licensing of distributions

Apache License, Version 2.0 (current)

The 2.0 version of the Apache License was approved by the ASF in 2004.

The goals of this license revision have been to reduce the number of frequently asked questions, to allow the license to be reusable without modification by any project (including non-ASF projects), to allow the license to be included by reference instead of listed in every file, to clarify the license on submission of contributions, to require a patent license on contributions that necessarily infringe the contributor's own patents, and to move comments regarding Apache and other inherited attribution notices to a location outside the license terms (the NOTICE file). All packages produced by the ASF are implicitly licensed under the Apache License, Version 2.0, unless otherwise explicitly stated.

[Apache License, Version 1.1 \(historic\)](#)

The 1.1 version of the Apache License was approved by the ASF in 2000. The primary change from the 1.0 license is in the 'advertising clause; derived products are no longer required to include attribution in their advertising materials, only in their documentation.

[Apache License, Version 1.0 \(historic\)](#)

This is the original Apache License which applies only to older versions of Apache packages (such as version 1.2 of the Web server).

Contributor License Agreements

The ASF desires that all contributors of ideas, code, or documentation to the Apache projects complete, sign, and submit (via postal mail, fax or e-mail) an Individual Contributor License Agreement (CLA) [PDF form]. The purpose of this agreement is to clearly define the terms under which intellectual property has been contributed to the ASF and thereby allow us to defend the project should there be a legal dispute regarding the software at some future time.

Conclusions

1. Open-Source software development which provides cost benefit features among other advantaged to the software development community.
2. A distinction should be made between free and Open Source software based on licensing agreements which again is not the same as freeware, which can be distributed for free.
3. The open source software can be a case for innovation which defies the principles of economics. It is interesting to note the motives such as "Being a good citizen" and "The problem is important to the company" play a role in the process of developers contributing to open source projects.
4. The Apache foundation was setup to provide legal setup assistance for open-source software developers. The Apache Web server is good example of the success of Open-source software which is being used by about 60% of web developers.

BERKELEY SOFTWARE DISTRIBUTION

The first BSD OS was released in 1977 in Berkeley, California, as part of a university student's project. BSD versions one to four were released between 1977 and 1980. Multiple versions of BSD were developed in parallel due to the open-source nature of the operating system. It was based on the original UNIX source code generated at Bell Labs rather than major software updates.

FreeBSD is a free and open-source UNIX operating system that is based on the Berkeley Software Distribution (BSD). The first version of the FreeBSD operating system was released in 1993. It was the most well-known open-source BSD OS in 2005 and is also responsible for more than three-quarters of all BSD systems installed with a simple and permissive license.

History of BSD

PDP-11 Beginnings

- ▶ The earliest distributions of UNIX from Bell Labs in the 1970s included the source code to the operating system, allowing researchers at universities to modify and extend UNIX.
- ▶ The first UNIX system at Berkeley was a PDP-11 installed in 1974, and the computer science department used it for research.
- ▶ In 1977 Bill Joy, assembled and sent out tapes of the first Berkeley Software Distribution (1BSD). 1BSD was an add on to Sixth Edition Unix rather than a complete operating system in its own right; its main components were a Pascal compiler and Joy's ex line editor.

The Second Berkeley Software Distribution (2BSD), released in 1978, included updated versions of the 1BSD software as well as two new programs. The vi text editor (a visual version of ex) and the C shell.

Later releases of 2BSD contained ports of changes to the VAX-based releases of BSD back to the PDP-11 architecture. 2.9BSD from 1983 included code from 4.1cBSD, and patches.

The most recent release, 2.11BSD, was first released in 1992, with maintenance updates from volunteers continuing until 2006.

BSD has been the base of a large number of operating systems. Most notable among these today is perhaps the major open source BSDs, FreeBSD, NetBSD and OpenBSD sometimes known as the BSDs-which have themselves spawned a number of children, including Dragonfly BSD, FreeSBIE, MirOS BSD, DesktopBSD, and PC-BSD.

Significant BSD Descendants

BSD has been the base of a large number of operating systems. Most notable among these today is the major open-source BSDs, FreeBSD,

NetBSD and OpenBSD sometimes known as the BSDs- including Dragonfly BSD, FreeBSD, MirOS BSD, DesktopBSD, and PC-BSD.

They are targeted at an array of systems for different purposes and are common in government facilities, universities and in commercial use. A number of commercial operating systems are also partly or wholly based on BSD or its descendants, including Sun's SunOS and Apple Inc.'s Mac OS X.

A significant Unix versions and Unix-like operating systems that descend from BSD includes:

- ▶ FreeBSD, a major open source effort focusing on performance and the x86 platform.
- ▶ NetBSD, an open source BSD with an emphasis on portability and clean design.
- ▶ OpenBSD, a 1995 fork of NetBSD, focuses on portability, standardization, correctness, proactive security and integrated cryptograph.
- ▶ DragonFly BSD, a fork of FreeBSD to follow alternative design, particularly related to SMP.
- ▶ PC-BSD and DesktopBSD, distributions. of FreeBSD with emphasis on ease of use and user friendly interfaces for the desktop/laptop PC user.
- ▶ Tru64 UNIX (formerly DEC OSF / 1 AXP or Digital UNIX), the port of OSF / 1 for DEC Alpha-based systems from DEC, Compaq and HP.
- ▶ Juniper Networks JunOS, the operating system for. Juniper routers, a customized version of FreeBSD, and a variety of other embedded operating systems.
- ▶ Apple Inc.'s Darwin, the core of Mac OS X; built on the XNU kernel (part Mach, part FreeBSD, part Apple-derived code) and a userland much of which comes from FreeBSD.
- ▶ Early versions of Sun Microsystems SunOS (up to StmOS 4.1.4),

an enhanced version of 4BSD for the Sun Motorola 68k-based Sun-2 and Sun-3 systems, SPARC-based systems, and x86-based Sun386i systems.

- DEC's Ultrix, the official version of Unix for its PDP-11, VAX, and DECstation systems.
- OSF/1, a microkernel-based UNIX developed by the Open Software Foundation, incorporating the Mach kernel and parts of 4BSD.
- DEMOS, a Soviet BSD clone.
- NeXT NEXTSTEP and OPENSTEP, based on the Mach kernel and 4BSD; the ancestor of Mac OS X.
- 386BSD, the first open source BSD-based operating system and the ancestor of most current BSD systems.
- BSD/OS, a (now defunct) proprietary BSD for PCs

Advantages:

1. It is a very mature and stable operating system.
2. It's released under the BSD license, allowing you to freely use, share, modify, and even sell the finished product.
3. It provides better performance.
4. It is mainly focused on security. It is one of the most secure systems available.
5. It has extensive documentation.

Disadvantages:

6. FreeBSD has a small community of supporters. This open-source software has a vibrant community.
7. There is a learning curve, and it takes much time and practice to understand completely.
8. There is just a small quantity of third-party software available for FreeBSD.

MOZILLA (FIREFOX)

The Mozilla project was created in 1998 with the release of the Netscape browser suite source code. By creating an open community, the Mozilla project had become larger than any one company.

People contributed to Mozilla in different ways, everyone was passionate about creating free software that would enable people to have a choice in how they experienced the internet.

After several years of development, Mozilla 1.0, the first major version, was released in 2002. This version featured many improvements to the browser, email client and other applications.

In 2003, the Mozilla project created the Mozilla Foundation, an independent non-profit organization supported by individual donors and a variety of companies. The new Mozilla Foundation continued the role of managing the daily operations of the project and also officially took on the role of promoting openness, innovation and opportunity on the internet.

By 2002, 90% of internet users were browsing with Internet Explorer. Not many people noticed at the time, but the first version of Phoenix (later renamed to Firefox) was also released by Mozilla community members that year with the goal of providing the best possible browsing experience to the widest possible set of people.

Firefox 1.0 was released in 2004 and became a big success — in less than a year, it was downloaded over 100 million times. New versions of Firefox have come out regularly by setting new records.

In 2013, Firefox OS was launched and also celebrated its 15th anniversary. The Mozilla community, together with other open source projects and other public benefit organizations, exists only because of the people who are engaged in making our common goals a reality.

Features:

- ▶ Firefox includes tabbed browsing,
- ▶ a spell checker,
- ▶ incremental find,
- ▶ live bookmarking,
- ▶ a download manager, and
- ▶ A search system that includes Google.

Functions can be added through more than 2,000 add-ons created by third party developers; the most popular include FoxyTunes (controls music players), Adblock Plus (ad blocker), StumbleUpon (website discovery),

DownThemAll! (download functions) and Web Developer (web tools).

Firefox runs on various versions of Windows, Mac OS X and Linux. Its current stable release is version 2.0.0.11. Firefox's source code is under the terms of the Mozilla tri- license as free and open source software. Mozilla was the official, public, original name of Mozilla Application Suite by the Mozilla Foundation.

Mozilla project

Mozilla Foundation

"Mozilla" is sometimes used to refer to the free software/open source project that was founded in order to create the next-generation Internet suite for Netscape. On July 15, 2003, the organization was formally registered as a non-profit organization, and became Mozilla Foundation. The foundation now creates and maintains the Mozilla Firefox browser and Mozilla Thunderbird e-mail application, among other software. The Mozilla trademark is held by the MozillaFoundation as of 2006.

1. Mozilla Corporation

On August 3, 2005, Mozilla Foundation announced the creation of Mozilla Corporation, a wholly owned for-profit taxable subsidiary of Mozilla Foundation, that primarily focuses on delivering Firefox to end users. It will also oversee marketing and sponsorship of the products.

2. Mozilla Messaging

On February 19, 2008, Mozilla Messaging was announced, which like Mozilla Corporation is a for-profit subsidiary of Mozilla Foundation. Its focus will be on Thunderbird and possibly developing software related to other internet communication media.

Security of Mozilla Firefox

- ▶ Firefox uses a sandbox security model, and limits scripts from accessing data from other websites based on the same origin policy.
- ▶ It uses SSL/TLS to protect communications with web servers using strong cryptography when using the https protocol.

- ▶ It also provides support for web applications to use smartcards for authentication purposes.
- ▶ The Mozilla Foundation offers a "bug bounty" to researchers who discover severe security holes in Firefox. Official guidelines for handling security vulnerabilities discourage early disclosure of vulnerabilities so as not to give potential attackers an advantage in creating exploits.
- ▶ Firefox has fewer and less severe publicly known unpatched security vulnerabilities than Internet Explorer improved security is often cited as a reason to switch from Internet Explorer to Firefox.
- ▶ In comparison, exploit code for critical security vulnerabilities in Firefox was available for 9 days before Mozilla shipped a patch to remedy the problem.
- ▶ A 2006 Symantec study showed that although Firefox had surpassed other browsers in the number of vendor-confirmed vulnerabilities, these vulnerabilities were patched far more quickly than those found in other browsers.
- ▶ Firefox still had fewer security vulnerabilities than Internet Explorer, as counted by security researchers. As of January 07, 2008, Firefox 2 has five security vulnerabilities unpatched, the most severe of which was rated "less critical".

WIKIPEDIA

- ▶ Wikipedia is a free online encyclopedia that anyone can add to or edit.
- ▶ The site was launched in early · 2001 and has since grown to include millions of articles in dozens of languages.
- ▶ Despite concerns about the quality of openly editable information, Wikipedia has become one of the most popular online resources- statistics put Wikipedia as the eighth most-visited Web site in the United States, behind sites such as Yahoo, Google, MySpace, and eBay.
- ▶ Article topics range from the very broad to the highly specific, and the site offers tools to organize information into various content areas- such as "academic disciplines" and "glossaries"-with numerous topic

breakdowns within each category. Each article contains any number of links to other Wikipedia articles or to external resources.

How Wikipedia works

- ▶ Through keyword searches, users locate and read articles, access photos and other graphics, and follow hyperlinks to resources related to the topic at hand.
- ▶ On certain Wikipedia pages, flags indicate if an article is out-of-date, for example, or contains no references, alerting users that the content might be less trustworthy than material on unflagged pages.
- ▶ The site also invites anyone to be a contributor. Although Wikipedia encourages contributors to create accounts, registration is not required.
- ▶ Users can edit or create entire articles, make changes to individual sections of articles, change the structure of an article, add images or links—all with the understanding that everything in Wikipedia can be changed by other users.
- ▶ The exceptions are articles that are "protected," a status assigned to entries that are targets of vandalism (intentionally posting inaccurate or defamatory information) or that frequently spawn ongoing disagreements among contributors.
- ▶ Wikipedia users with administrator rights have the authority to assign or remove protection status, delete pages, and block individual users from editing articles. Protected (and semi-protected) pages require users to meet certain conditions to make edits. To change a page, for example, a user must have been registered for at least four days.
- ▶ A Discussion tab for each article provides a venue for users to discuss that article, its history and structure, and ways to improve it. Many articles (or sections of articles) are inherently contentious, and the Discussion tab is a place where users can have a side bar conversation about why certain edits to the article have been made or what information might expand an article's value without expressing a bias.

- The Discussion tab serves to discuss between contributors are disconnected from one another.

Joomla

Joomla! is an award-winning Content Management System (CMS) that will help you build websites and other powerful online applications. Joomla! Is an open-source solution that is freely available to everybody. Joomla! is a free, open-source content management system for publishing content on the worldwide web and intranets.

Joomla! includes features such as page caching to improve performance, RSS feeds, printable versions of pages, news flashes, blogs, polls, website searching, and language internationalization. Joomla! Is licensed under the GPL, and is the result of a fork of Mambo. It is written with the PHP programming language and uses the MySQL database by default.

Developed by	The OSM Development Team
Latest release	1.5.1 / February 8, 2008
OS	Cross-platform
Genre	Content Management System
License	GPL
Website	http ://www.joomla.org/

Joomla for End Users

The basic Joomla! Package is designed to be easy to install, even for non-programmers. Most people have no trouble getting our software up and running, and there is plenty of support available for newbie's. We have a growing, active community of more than 150,000 friendly users and developers on our forums eager to help.

Once Joomla! is installed and running, it is simple for even non-technical users to add or edit content, update images, and to manage the critical data that makes your company or organization go. Anybody with basic word processing skills can easily learn to manage a Joomla! Site.

Via a simple, browser-based interface you will be able to easily add new press releases or news items, manage staff pages, job listings, product images, and create an unlimited amount of sections or content pages on your site. You can try our simple demo to get quick taste of what Joomla!

is all about.

Joomla for Developers

Joomla! offers a powerful application framework that makes it easy for developers to create sophisticated add-ons that extend the power of Joomla! into virtually unlimited directions. Using the core framework, developers can easily build:

- Integrated e-commerce systems
- Inventory control systems
- Data reporting tools
- Custom product catalogs
- Complex business directories
- Reservation systems
- Communication tools
- Application bridges or any kind of application to suit a unique need.

If your company or organization hires a third-party Joomla! developer or builds in-house software using the Joomla! Framework, you are building on an open platform that does not tie you to anyone developer or to a proprietary, closed application.

Advantages of Joomla

- It is a freely available open-source platform.
- It is quite simple to install and set up.
- Almost all the browsers support it by default.
- It has an easy menu creation tool.
- It can be migrated to any server, and most of them work with any database.

Disadvantages of Joomla

- ▶ It is quite complex and makes it difficult to get custom designs without developer experience.
- ▶ It is not much SEO (Search Engine Optimization) friendly.
- ▶ It makes websites heavy to load and run, resulting in more time.

The GNU Compiler Collection

The GNU Compiler Collection (usually shortened to GCC) is a set of compilers produced for various programming languages by the GNU

GNU COMPILER COLLECTION

Developed by	GNU Project
Latest release	4.2.3 / 1 February, 2008
OS	Cross-platform
Platform	GNU
Genre	Compiler
License	GNU General Public License, GNU Lesser General Public License
Website	http ://gcc.gnu.org/

Project.

GCC is a key component of the GNU toolchain. As well as being the official compiler of the GNU system, GCC has been adopted as the standard compiler by most other modern Unix- like computer operating systems, including Linux, the BSD family and Mac OS X.

GCC has been ported to a wide variety of computer architectures, and is widely deployed as a tool in commercial, proprietary and closed source software development environments. GCC is also used in popular embedded platforms like Symbian, PlayStation and Sega Dreamcast.

Originally named the GNU C Compiler, because it only handled the C programming language, GCC 1.0 was released in 1987, and the compiler was extended to compile C++ in December of that year. Front ends were later developed for Fortran, Pascal, Objective C, Java, and Ada, among others.

Distributed by the Free Software Foundation (FSF) under the terms

of the GNU General Public License (GNU GPL) and the GNU Lesser General Public License (GNU LGPL), GCC is free software.

The standard compiler release 4.2 includes front ends for: C, C++ (G++), Java (GCD), Ada (GNAT), Objective-C, Objective-C++, and Fortran (GFortran). Also available, but not in standard are: Modula-2, Modula-3, Pascal, PL/I, D (dgcc), Mercury, VHOL (GHDL).

LIBRE OFFICE:

Libre Office is also an open-source software, it used for typing the letters, calculating the values, and presenting like power point. Its equivalent to the MS-Office, like MS- Word, MS-Excel and MSPowerpoint, suppose if we want use the MS- Office we need to pay monthly some amount to the Micro Soft Corporation, instead of this we can use this free software. This also behave same like MS - Office. Web Defination: LibreOffice is a powerful and free office suite, a successor to OpenOffice(.org), used by millions of people around the world. Its clean interface and feature-rich tools help you unleash your creativity and enhance your productivity. LibreOffice includes several applications that make it the most versatile Free and Open Source office suite on the market: Writer (word processing), Calc (spreadsheets), Impress (presentations), Draw (vector graphics and flowcharts), Base (databases), and Math.

OPENOFFICE

OpenOffice.org (OO.o or OOo) is an office application suite available for a number of different computer operating systems.

1. **Writer:** A word processor similar in look and feel to Microsoft Word and offering a comparable range of functions and tools. It also includes the ability to export Portable Document Format (PDF) files with no additional software, and can also function as a WYSIWYG editor for creating and editing web pages. One important difference between Writer and Microsoft Word is that in Writer,

many functions and number formats from Calc (below) are available in Writer's tables.

2. **Calc** A spreadsheet similar to Microsoft Excel with a roughly equivalent range of features. Calc provides a number of features not present in Excel, including a system which automatically defines series for graphing, based on the layout of the user's data. Calc is also capable of writing spreadsheets directly as a PDF file.
3. **Impress** A presentation program similar to Microsoft PowerPoint. It can export presentations to Adobe Flash (SWF) files allowing them to be played on any computer with the Flash player installed. It also includes the ability to create PDF files, and the ability to read Microsoft PowerPoint's .ppt format. Impress suffers from a lack of readymade presentation designs. templates are readily available on the Internet.
4. **Base** A database program similar to Microsoft Access. Base allows the creation and manipulation of databases, and the building of forms and reports to provide easy access to data for end-users. As with Access, Base may be used as a front-end to a number of different database systems, including Access databases (JET), ODBC data sources and MySQL/ PostgreSQL.
5. **Draw** A vector graphics editor comparable in features to early versions of CorelDRAW. It features versatile "connectors" between shapes, which are available in a range of line styles and facilitate building drawings such as flowcharts. It has similar features to Desktop publishing software such as Scribus and Microsoft Publisher.
6. **Math** A tool for creating and editing mathematical formulae, similar to Microsoft Equation Editor. Formulae can be embedded inside other OpenOffice.org documents, such as those created by Writer. It supports multiple fonts and can export to PDF.
